



Conversation messages

Render a transcript from host-owned messages and actions.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-conversation-messages



When this library is useful

Render a transcript from host-owned messages and actions.

01

Display roles

Use user, assistant and status bubble variants.

02

Reveal a response

Render updated Markdown while a generation is active.

03

Compose evidence

Place stages, citations and attachments beside the answer.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Bubble layout, Markdown display and action controls.



The public contract to start with

AssistantMessageBubble · selected surface; full API index is in the bundle.

Input or operation	Contract
text (optional)	string
isStreaming (optional)	boolean
afterContent (optional)	ReactNode
onAttachmentClick (optional)	((attachment: DisplayAttachment) => void)



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
function MessageDemo() {  
  return <AssistantMessageBubble text="**Demo:** evidence collected."  
    isStreaming={false} deploymentDisplayName="Demo assistant"  
    afterContent={<StagesPanel stages={[]} isStreaming={false} />} />;  
}
```

Full example: [examples/conversation-messages.tsx](#) · uses [@epam/ai-dial-conversation-messages](#)



Wire a realistic host workflow

Keep conversation messages focused on its public responsibility.

01

Prepare a message

The host supplies response text and streaming state.

02

Compose evidence

Insert StagesPanel through the afterContent slot.

03

Connect actions

Keep regeneration, feedback and persistence in the host.



Available customization

01

Extension slots

beforeContent and
afterContent add skill
chips or stage panels.

02

Host URL rewriting

markdownUrlTransform
maps resource
references before
rendering.

03

Transcript controls

Supply MessageActions
props and localized
action labels.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	chat/components/ConversationView/ConversationMessageItem.tsx
Test evidence	calls onEdit when Edit button is clicked
Test evidence	marks the bubble that wraps the message text



Constraints that affect integration

01

Streaming is visual

isStreaming does not open a network stream.

02

Status messages

Shared
MessageRole.Status is a UI event, not an upstream author role.

03

Source integrations

Citation components and canvas actions require host wiring.